Computer Files

Information stored in a computer is organised into files.

A computer file is the basic unit of information stored in the computer's memory. In modern systems, these are contiguous bytes of memory that hold related data. Each computer file can contain different types of data depending on its intended purpose. For example, a computer file can contain text, audio, video, image or other binary data. The format of the file is determined by its content but can also be indicated by its file extension.

Multiple computer files can be grouped together and organised into folders (also known as directories). A computer folder is a container for computer files and can contain other folders (known as sub-folders). A folder that contains other folders is known as the parent folder. The top-most folder is known as the root folder. Computer folders are organised using a file system which manages how computer files are stored and accessed.

File Paths and the Current Working Folder

A computer program can be used to create, manipulate or remove computer files. Computer programs have a current working folder which is the folder in which the computer program runs. A computer program can access files using the file path for a computer file. A file path indicates the location of a file in a file system. The file path can be absolute or relative. An absolute path points to the same location in the file system regardless of the current working folder. A relative path starts from the current working folder and helps avoid the need for an absolute path.

As an example, consider the following folder structure:

```
data
|
|-- staff
|   |__ prins.txt
|   |__ prins.png
|
|-- students
|   |__ attendance.csv
```

In the example above, we have the root folder data which contains the sub-folders staff and students. The folder staff contains the files prins.txt and prins.png, The folder students contains the files attendance.csv.

The absolute path to the root folder is as follows:

```
data
```

or, more commonly,

```
/
```

Where the forward slash is a short way to refer to the root folder.

The absolute path to the file prins.txt is as follows:

```
data/staff/prins.txt
```

?

The absolute path to the file attendance.csv is as follows:

```
data/students/attendance.csv
```

If the current working directory is staff then the relative path to the file prins.txt is as follows:

```
prins.txt
```

or

```
./prins.txt
```

Where the single dot (.) refers to the current working directory.

If the current working directory is staff then the relative path to the file attedance.csv is as follows:

```
../students/attendance.csv
```

Where the two dots (..) refers to the parent folder. Similarly, ../.. refers to the parent of the parent (grandparent) folder.

File Formats

Many different file formats exist for storing different types of information in files. For the purpose of this module, we will consider some common file formats including TXT, CSV, JSON and images.

A TXT file has the extension .txt and contains unformatted text. An example of TXT file is shown below:

sample.txt

```
This is a text file with some sample data.
```

A CSV file has the extension .csv. The acronym CSV stands for comma separated values and indicates how text is stored in the file. Often these files represent data from a spreadsheet or database where each row is a record and consists of one or more values separated by commas. The first row in a CSV document often contains headers which indicate what is represented by each column of values in the document. This format is very popular and is useful for transferring tabular data between programs. An example of a CSV document is shown below:

movies.csv

```
Film,Genre,Year
Tangled,Animation,2010
Gnomeo and Juliet,Animation,2011
WALL-E,Animation,2008
```

JSON stands for JavaScript Object Notation and is an independent text-based interchange format. Files in this format have the extension .json and are often used to exchange data between a web application and a server. These files contain simple data structures (similar to dictionaries) that can be processed efficiently and flexibly. An example of a JSON document is shown below:

songs.json

```
[
  {
    "song_id": "1",
    "song_title": "Zombie",
    "artist_id": "14",
    "album_id": 1,
    "duration": 301.123,
```

```
    "year": "1994"
  },
  {
    "song_id": "2",
    "song_title": "In The End",
    "artist_id": "35",
    "album_id": 9,
    "duration": 271.405,
    "year": 2002
  }
]
```

Python and Files

Python includes modules that allow us to work with files. Using python, we can create, read, open, write and delete files.

Python treats files as one of two categories - either as text files or binary files.

A text file is a file that contains data stored using a text-based format. Text files can be plain (i.e. contain no formatting) or contain text with formatting. Some examples of plain text files include files with the extensions .txt, .csv and .json. Some examples of text files that contain formatting include .docx and .xls.

A binary file is a file that contains data stored using a format other than a text-based format. For example, audio, video and image files are binary files and can be processed as such by Python.

Python and File Handling - TXT Files

The key function in python to work with files is open. This takes two parameters, the file name and the mode. The file name may be an absolute or relative path. The mode indicates how the file should be opened. There are 4 modes for opening a file:

Read - This is the default and can be selected by providing the parameter "r" for mode. In this mode, the file is opened for reading. If the file does not exist then an error is generated.

Append - This is selected by providing the parameter "a" for mode. In this mode, the file is opened for appending. If the file does not exist then it will first be created.

Write - This is selected by providing the parameter "w" for mode. In this mode, the file is opened for writing. If the file does not exist then it will first be created.

Create - This is selected by providing the parameter "x" for mode. In this mode, the file is created. If the file already exists then an error will be generated.

Additionally, the file can be handled as either a text file or a binary file by supplying the values "t" or "b"for the mode respectively. By default, a file is handled as a text file.

For example, if we wish to open an existing text file for reading we can use any of the following:

```
file = open("data.txt", "rt")
```

or, as text file is the default type of file, we have:

```
file = open("data.txt", "r")
```

or, as read is also the default mode, we have:

```
file = open("data.txt")
```

Similarly, if we wish to create and write to a new binary file, we can do the following:

```
file = open("output.png", "wb")
```

Once a file has been processed, it should be closed to free resources. This can be done using the method close of file.

```
file.close()
```

Python and File Handling - CSV Files

When working with CSV files, we can use the module csv to simplify file handling. This module provides us with a CSV reader that allows us to retrieve a row of data one at a time from the file. Each row is a sequence of data that can be accessed using an index number (e.g. row[0] for the first value, row[1] for the second value and so on).

For example, if we wish to open a CSV file for reading and display the first column of each row then we can do the following:

```
import csv
with open("data.csv") as csv_file:
    csv_reader = csv.reader(csv_file, delimiter=',', quotechar='"')
    for row in csv_reader:
        print(row[0])
```

Python and File Handling - JSON Files

Similarly, when working with JSON, we can use the module json to load and process JSON. The module provides the method load that allows us to easily load JSON data from a file into a Python dictionary.

```
import json

with open("data.json") as json_file:
  data = json.load(json_file)

print(data)
```

It is often useful to write JSON to a file. This can be done using the method dump.

```
import json

json_data = {
    "name": "Prins",
    "job": "Lecturer"
}

json_file = open("output.json", "w")
json.dump(json_data, json_file, indent = 4)

json_file.close()
```

Alternatively, the methods loads and dumps can be used when working with strings rather than files.

Python and Remote Files

In real-world (web) applications, we often need to access and send files from and to remote (web) locations. We can use the module requests to allow us to send or received files from a remote file server.

We can get data (e.g. a JSON file) from a server using the method get:

```
import requests

response = requests.get("https://somesite.com/data.json")
print(response.json())
```

We can send data (e.g. JSON) to a server using the method post:

```
import json
import requests

data = {
    "name": "Prins",
    "job": "Lecturer"
}

json_data = json.dumps(data)
requests.post("https://somesite.com/post", json=json_data)
```

Copyright © 2023 Prins Butt, All rights reserved.

Last modified: Saturday, 15 November 2025, 2:27 PM

◀ Extra Help

Jump to...

2 - Text Files ▶

Dashboard / Problem Solving Through Programming (QHO444) / QHO444 Materials / 1 - Data Files

Study

Succeed@Solent
Referencing
Subject Guides
Library
Ethics

Organise

Email
Timetables
Term Dates
Portal

Support

Student Hub
Technical Support
Printing
Extenuating Circumstances

Solent Futures

Solent Careers Online
CV Help
Placements

Disclaimer | Terms & Conditions | Cookies | Accessibility Statement |

© Solent University